

```
import pandas as pd
```

```
dataset=pd.read_csv("Placement.csv")
```

```
dataset
```

	sl_no	gender	ssc_p	ssc_b	hsc_p	hsc_b	hsc_s	degree_p
0	1	M	67.00	Others	91.00	Others	Commerce	58.00
1	2	M	79.33	Central	78.33	Others	Science	77.48
2	3	M	65.00	Central	68.00	Central	Arts	64.00
3	4	M	56.00	Central	52.00	Central	Science	52.00
4	5	M	85.80	Central	73.60	Central	Commerce	73.30
..	...	...	...	...	...	...	...	...
210	211	M	80.60	Others	82.00	Others	Commerce	77.60
211	212	M	58.00	Others	60.00	Others	Science	72.00
212	213	M	67.00	Others	67.00	Others	Commerce	73.00
213	214	F	74.00	Others	66.00	Others	Commerce	58.00
214	215	M	62.00	Central	58.00	Others	Science	53.00

	degree_t	workex	etest_p	specialisation	mba_p	status
salary						
0	Sci&Tech	No	55.0	Mkt&HR	58.80	Placed
270000.0						
1	Sci&Tech	Yes	86.5	Mkt&Fin	66.28	Placed
200000.0						
2	Comm&Mgmt	No	75.0	Mkt&Fin	57.80	Placed
250000.0						
3	Sci&Tech	No	66.0	Mkt&HR	59.43	Not Placed
NaN						
4	Comm&Mgmt	No	96.8	Mkt&Fin	55.50	Placed
425000.0						
..	...	...	...	...	...	...
...						
210	Comm&Mgmt	No	91.0	Mkt&Fin	74.49	Placed
400000.0						
211	Sci&Tech	No	74.0	Mkt&Fin	53.62	Placed
275000.0						
212	Comm&Mgmt	Yes	59.0	Mkt&Fin	69.72	Placed
295000.0						

213	Comm&Mgmt	No	70.0	Mkt&HR	60.23	Placed
204000.0						
214	Comm&Mgmt	No	89.0	Mkt&HR	60.22	Not Placed
NaN						

[215 rows x 15 columns]

dataset.info()

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 215 entries, 0 to 214

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	sl_no	215 non-null	int64
1	gender	215 non-null	object
2	ssc_p	215 non-null	float64
3	ssc_b	215 non-null	object
4	hsc_p	215 non-null	float64
5	hsc_b	215 non-null	object
6	hsc_s	215 non-null	object
7	degree_p	215 non-null	float64
8	degree_t	215 non-null	object
9	workex	215 non-null	object
10	etest_p	215 non-null	float64
11	specialisation	215 non-null	object
12	mba_p	215 non-null	float64
13	status	215 non-null	object
14	salary	148 non-null	float64

dtypes: float64(6), int64(1), object(8)

memory usage: 25.3+ KB

```
#Quan=["ssc_p","hsc_p","etest_p","degree_p","mba_p","salary"]
```

```
#Qual=["gender","ssc_b","hsc_b","degree_t"]
```

dataset.columns

```
Index(['sl_no', 'gender', 'ssc_p', 'ssc_b', 'hsc_p', 'hsc_b', 'hsc_s',
       'degree_p', 'degree_t', 'workex', 'etest_p', 'specialisation',
       'mba_p',
       'status', 'salary'],
      dtype='object')
```

dir(dataset)

```
['T',
 '_AXIS_LEN',
 '_AXIS_ORDERS',
 '_AXIS_TO_AXIS_NUMBER',
 '_HANDLED_TYPES',
 '__abs__',
```

```
'__add__',
'__and__',
'__annotations__',
'__array__',
'__array_priority__',
'__array_ufunc__',
'__bool__',
'__class__',
'__contains__',
'__copy__',
'__dataframe__',
'__dataframe_consortium_standard__',
'__deepcopy__',
'__delattr__',
'__delitem__',
'__dict__',
'__dir__',
'__divmod__',
'__doc__',
'__eq__',
'__finalize__',
'__floordiv__',
'__format__',
'__ge__',
'__getattr__',
'__getattribute__',
'__getitem__',
'__getstate__',
'__gt__',
'__hash__',
'__iadd__',
'__iand__',
'__ifloordiv__',
'__imod__',
'__imul__',
'__init__',
'__init_subclass__',
'__invert__',
'__ior__',
'__ipow__',
'__isub__',
'__iter__',
'__itruediv__',
'__ixor__',
'__le__',
'__len__',
'__lt__',
'__matmul__',
'__mod__',
```

```
'__module__',
'__mul__',
'__ne__',
'__neg__',
'__new__',
'__nonzero__',
'__or__',
'__pandas_priority__',
'__pos__',
'__pow__',
'__radd__',
'__rand__',
'__rdivmod__',
'__reduce__',
'__reduce_ex__',
'__repr__',
'__rfloordiv__',
'__rmatmul__',
'__rmod__',
'__rmul__',
'__ror__',
'__round__',
'__rpow__',
'__rsub__',
'__rtruediv__',
'__rxor__',
'__setattr__',
'__setitem__',
'__setstate__',
'__sizeof__',
'__str__',
'__sub__',
'__subclasshook__',
'__truediv__',
'__weakref__',
'__xor__',
'accessors',
'accum_func',
'agg_examples_doc',
'agg_see_also_doc',
'_align_for_op',
'_align_frame',
'_align_series',
'_append',
'_arith_method',
'_arith_method_with_reindex',
'_as_manager',
'_attrs',
'_box_col_values',
```

```
'_can_fast_transpose',
'_check_inplace_and_allows_duplicate_labels',
'_check_is_chained_assignment_possible',
'_check_label_or_level_ambiguity',
'_check_setitem_copy',
'_clear_item_cache',
'_clip_with_one_bound',
'_clip_with_scalar',
'_cmp_method',
'_combine_frame',
'_consolidate',
'_consolidate_inplace',
'_construct_axes_dict',
'_construct_result',
'_constructor',
'_constructor_from_mgr',
'_constructor_sliced',
'_constructor_sliced_from_mgr',
'_create_data_for_split_and_tight_to_dict',
'_data',
'_deprecate_downcast',
'_dir_additions',
'_dir_deletions',
'_dispatch_frame_op',
'_drop_axis',
'_drop_labels_or_levels',
'_ensure_valid_index',
'_find_valid_index',
'_flags',
'_flex_arith_method',
'_flex_cmp_method',
'_from_arrays',
'_from_mgr',
'_get_agg_axis',
'_get_axis',
'_get_axis_name',
'_get_axis_number',
'_get_axis_resolvers',
'_get_block_manager_axis',
'_get_bool_data',
'_get_cleaned_column_resolvers',
'_get_column_array',
'_get_index_resolvers',
'_get_item_cache',
'_get_label_or_level_values',
'_get_numeric_data',
'_get_value',
'_getitem_bool_array',
'_getitem_multilevel',
```

```
'_getitem_nocopy',
'_getitem_slice',
'_getitem',
'_hidden_attrs',
'_indexed_same',
'_info_axis',
'_info_axis_name',
'_info_axis_number',
'_info_repr',
'_init_mgr',
'_inplace_method',
'_internal_names',
'_internal_names_set',
'_is_copy',
'_is_homogeneous_type',
'_is_label_or_level_reference',
'_is_label_reference',
'_is_level_reference',
'_is_mixed_type',
'_is_view',
'_iset_item',
'_iset_item_mgr',
'_iset_not_inplace',
'_item_cache',
'_iter_column_arrays',
'_ixs',
'_logical_func',
'_logical_method',
'_maybe_align_series_as_frame',
'_maybe_cache_changed',
'_maybe_update_cacher',
'_metadata',
'_mgr',
'_min_count_stat_function',
'_needs_reindex_multi',
'_pad_or_backfill',
'_protect_consolidate',
'_reduce',
'_reduce_axis1',
'_reindex_axes',
'_reindex_multi',
'_reindex_with_indexers',
'_rename',
'_replace_columnwise',
'_repr_data_resource',
'_repr_fits_horizontal',
'_repr_fits_vertical',
'_repr_html',
'_repr_latex',
```

```
'_reset_cache',
'_reset_cacher',
'_sanitize_column',
'_series',
'_set_axis',
'_set_axis_name',
'_set_axis_nocheck',
'_set_is_copy',
'_set_item',
'_set_item_frame_value',
'_set_item_mgr',
'_set_value',
'_setitem_array',
'_setitem_frame',
'_setitem_slice',
'_shift_with_freq',
'_should_reindex_frame_op',
'_slice',
'_sliced_from_mgr',
'_stat_function',
'_stat_function_ddof',
'_take_with_is_copy',
'_to_dict_of_blocks',
'_to_latex_via_styler',
'_typ',
'_update_inplace',
'_validate_dtype',
'_values',
'_where',
'abs',
'add',
'add_prefix',
'add_suffix',
'agg',
'aggregate',
'align',
'all',
'any',
'apply',
'applymap',
'asfreq',
'asof',
'assign',
'astype',
'at',
'at_time',
'attrs',
'axes',
'backfill',
```

```
'between_time',
'bfill',
'bool',
'boxplot',
'clip',
'columns',
'combine',
'combine_first',
'compare',
'convert_dtypes',
'copy',
'corr',
'corrwith',
'count',
'cov',
'cummax',
'cummin',
'cumprod',
'cumsum',
'degree_p',
'degree_t',
'describe',
'diff',
'div',
'divide',
'dot',
'drop',
'drop_duplicates',
'droplevel',
'dropna',
'dtypes',
'duplicated',
'empty',
'eq',
'equals',
'etest_p',
'eval',
'ewm',
'expanding',
'explode',
'ffill',
'fillna',
'filter',
'first',
'first_valid_index',
'flags',
'floordiv',
'from_dict',
'from_records',
```



```
'ge',  
'gender',  
'get',  
'groupby',  
'gt',  
'head',  
'hist',  
'hsc_b',  
'hsc_p',  
'hsc_s',  
'iat',  
'idxmax',  
'idxmin',  
'iloc',  
'index',  
'infer_objects',  
'info',  
'insert',  
'interpolate',  
'isetitem',  
'isin',  
'isna',  
'isnull',  
'items',  
'iterrows',  
'itertuples',  
'join',  
'keys',  
'kurt',  
'kurtosis',  
'last',  
'last_valid_index',  
'le',  
'loc',  
'lt',  
'map',  
'mask',  
'max',  
'mba_p',  
'mean',  
'median',  
'melt',  
'memory_usage',  
'merge',  
'min',  
'mod',  
'mode',  
'mul',  
'multiply',
```

```
'ndim',  
'ne',  
'nlargest',  
'notna',  
'notnull',  
'nsmallest',  
'nunique',  
'pad',  
'pct_change',  
'pipe',  
'pivot',  
'pivot_table',  
'plot',  
'pop',  
'pow',  
'prod',  
'product',  
'quantile',  
'query',  
'radd',  
'rank',  
'rdiv',  
'reindex',  
'reindex_like',  
'rename',  
'rename_axis',  
'reorder_levels',  
'replace',  
'resample',  
'reset_index',  
'rfloordiv',  
'rmod',  
'rmul',  
'rolling',  
'round',  
'rpow',  
'rsub',  
'rtruediv',  
'salary',  
'sample',  
'select_dtypes',  
'sem',  
'set_axis',  
'set_flags',  
'set_index',  
'shape',  
'shift',  
'size',  
'skew',
```

```
'sl_no',
'sort_index',
'sort_values',
'specialisation',
'squeeze',
'ssc_b',
'ssc_p',
'stack',
'status',
'std',
'style',
'sub',
'subtract',
'sum',
'swapaxes',
'swaplevel',
'tail',
'take',
'to_clipboard',
'to_csv',
'to_dict',
'to_excel',
'to_feather',
'to_gbq',
'to_hdf',
'to_html',
'to_json',
'to_latex',
'to_markdown',
'to_numpy',
'to_orc',
'to_parquet',
'to_period',
'to_pickle',
'to_records',
'to_sql',
'to_stata',
'to_string',
'to_timestamp',
'to_xarray',
'to_xml',
'transform',
'transpose',
'truediv',
'truncate',
'tz_convert',
'tz_localize',
'unstack',
'update',
'value_counts',
```

```
'values',  
'var',  
'where',  
'workex',  
'xs']
```

```
dataset["ssc_p"].dtypes
```

```
dtype('float64')
```

```
for columnName in dataset.columns:  
    print(columnName)
```

```
sl_no  
gender  
ssc_p  
ssc_b  
hsc_p  
hsc_b  
hsc_s  
degree_p  
degree_t  
workex  
etest_p  
specialisation  
mba_p  
status  
salary
```

```
if(dataset["ssc_p"].dtype=='0'):  
    print("qual")  
else:  
    print("quan")
```

```
quan
```

```
quan=[]  
qual=[]  
for columnName in dataset.columns:  
    #print(columnName)  
    if(dataset[columnName].dtype=='0'):  
        #print("qual")  
        qual.append(columnName)  
    else:  
        #print("quan")  
        quan.append(columnName)
```

```
quan
```

```
['sl_no', 'ssc_p', 'hsc_p', 'degree_p', 'etest_p', 'mba_p', 'salary']
```

```

qual
['gender',
'ssc_b',
'hsc_b',
'hsc_s',
'degree_t',
'workex',
'specialisation',
'status']

def quanQual(dataset):
    quan=[]
    qual=[]
    for columnName in dataset.columns:
        #print(columnName)
        if(dataset[columnName].dtype=='0'):
            #print("qual")
            qual.append(columnName)
        else:
            #print("quan")
            quan.append(columnName)
    return quan,qual

quanQual(dataset)

(['sl_no', 'ssc_p', 'hsc_p', 'degree_p', 'etest_p', 'mba_p',
'salary'],
 ['gender',
'ssc_b',
'hsc_b',
'hsc_s',
'degree_t',
'workex',
'specialisation',
'status'])

quan,qual=quanQual(dataset)

quan
['sl_no', 'ssc_p', 'hsc_p', 'degree_p', 'etest_p', 'mba_p', 'salary']

qual
['gender',
'ssc_b',
'hsc_b',

```

```
'hsc_s',
'degree_t',
'workex',
'specialisation',
'status']
```

dataset

	sl_no	gender	ssc_p	ssc_b	hsc_p	hsc_b	hsc_s	degree_p
\								
0	1	M	67.00	Others	91.00	Others	Commerce	58.00
1	2	M	79.33	Central	78.33	Others	Science	77.48
2	3	M	65.00	Central	68.00	Central	Arts	64.00
3	4	M	56.00	Central	52.00	Central	Science	52.00
4	5	M	85.80	Central	73.60	Central	Commerce	73.30
...	...	...	...	...	...	...	...	...
210	211	M	80.60	Others	82.00	Others	Commerce	77.60
211	212	M	58.00	Others	60.00	Others	Science	72.00
212	213	M	67.00	Others	67.00	Others	Commerce	73.00
213	214	F	74.00	Others	66.00	Others	Commerce	58.00
214	215	M	62.00	Central	58.00	Others	Science	53.00

	degree_t	workex	etest_p	specialisation	mba_p	status
salary						
0	Sci&Tech	No	55.0	Mkt&HR	58.80	Placed
270000.0						
1	Sci&Tech	Yes	86.5	Mkt&Fin	66.28	Placed
200000.0						
2	Comm&Mgmt	No	75.0	Mkt&Fin	57.80	Placed
250000.0						
3	Sci&Tech	No	66.0	Mkt&HR	59.43	Not Placed
NaN						
4	Comm&Mgmt	No	96.8	Mkt&Fin	55.50	Placed
425000.0						
...	...	...	...	...	...	...
...						
210	Comm&Mgmt	No	91.0	Mkt&Fin	74.49	Placed
400000.0						
211	Sci&Tech	No	74.0	Mkt&Fin	53.62	Placed
275000.0						

212	Comm&Mgmt	Yes	59.0	Mkt&Fin	69.72	Placed
295000.0						
213	Comm&Mgmt	No	70.0	Mkt&HR	60.23	Placed
204000.0						
214	Comm&Mgmt	No	89.0	Mkt&HR	60.22	Not Placed
NaN						

[215 rows x 15 columns]

Central tendency

```
dataset["ssc_p"].mean()
```

67.30339534883721

```
dataset["ssc_p"].median()
```

67.0

```
dataset["ssc_p"].mode()[0]
```

62.0

```
dataset.describe()
```

	sl_no	ssc_p	hsc_p	degree_p	etest_p
mba_p \					
count	215.000000	215.000000	215.000000	215.000000	215.000000
mean	108.000000	67.303395	66.333163	66.370186	72.100558
std	62.209324	10.827205	10.897509	7.358743	13.275956
min	1.000000	40.890000	37.000000	50.000000	50.000000
25%	54.500000	60.600000	60.900000	61.000000	60.000000
50%	108.000000	67.000000	65.000000	66.000000	71.000000
75%	161.500000	75.700000	73.000000	72.000000	83.500000
max	215.000000	89.400000	97.700000	91.000000	98.000000

	salary
count	148.000000
mean	288655.405405
std	93457.452420
min	200000.000000

25%	240000.000000
50%	265000.000000
75%	300000.000000
max	940000.000000

```
descriptive=pd.DataFrame()
```

```
descriptive=pd.DataFrame(index=["Mean","Median","Mode"],columns=quan)
```

```
descriptive
```

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p	salary
Mean	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Median	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Mode	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
descriptive["hsc_p"]["Median"]=0
```

```
descriptive=pd.DataFrame(index=["Mean","Median","Mode"],columns=quan)
```

```
for columnName in quan:
    descriptive[columnName]["Mean"]=dataset[columnName].mean()
```

```
descriptive
```

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p
Mean	108.0	67.303395	66.333163	66.370186	72.100558	62.278186
Median	NaN	NaN	NaN	NaN	NaN	NaN
Mode	NaN	NaN	NaN	NaN	NaN	NaN

	salary
Mean	288655.405405
Median	NaN
Mode	NaN

```
descriptive=pd.DataFrame(index=["Mean","Median","Mode"],columns=quan)
```

```
for columnName in quan:
    descriptive[columnName]["Mean"]=dataset[columnName].mean()
    descriptive[columnName]["Median"]=dataset[columnName].median()
```

```
descriptive
```

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p
Mean	108.0	67.303395	66.333163	66.370186	72.100558	62.278186
Median	108.0	67.0	65.0	66.0	71.0	62.0
Mode	NaN	NaN	NaN	NaN	NaN	NaN

	salary
Mean	288655.405405
Median	265000.0
Mode	NaN

```
descriptive=pd.DataFrame(index=["Mean","Median","Mode"],columns=quan)
for columnName in quan:
    descriptive[columnName]["Mean"]=dataset[columnName].mean()
    descriptive[columnName]["Median"]=dataset[columnName].median()
    descriptive[columnName]["Mode"]=dataset[columnName].mode()[0]
```

descriptive

	sl_no	ssc_p	hsc_p	degree_p	etest_p	
mba_p \						
Mean	108.0	67.303395	66.333163	66.370186	72.100558	62.278186
Median	108.0	67.0	65.0	66.0	71.0	62.0
Mode	1	62.0	63.0	65.0	60.0	56.7

	salary
Mean	288655.405405
Median	265000.0
Mode	300000.0

percentile

```
import numpy as np
```

```
np.percentile(dataset["ssc_p"],100)
```

89.4

```
descriptive=pd.DataFrame(index=["Mean","Median","Mode","Q1:25%","Q2:50%",
                                "Q3:75%","99%","Q4:100%"],columns=quan)
```

```
for columnName in quan:
    descriptive[columnName]["Mean"]=dataset[columnName].mean()
    descriptive[columnName]["Median"]=dataset[columnName].median()
    descriptive[columnName]["Mode"]=dataset[columnName].mode()[0]
    descriptive[columnName]["Q1:25%"]=dataset.describe()[columnName]
    descriptive[columnName]["Q2:50%"]=dataset.describe()[columnName]
    descriptive[columnName]["Q3:75%"]=dataset.describe()[columnName]
    descriptive[columnName]["Q4:100%"]=dataset.describe()[columnName]
```

```

        descriptive[columnName]["Q3:75%"]=dataset.describe()[columnName]
["75%"]
        descriptive[columnName]
["99%"]=np.percentile(dataset[columnName],99)
        descriptive[columnName]["Q4:100%"]=dataset.describe()[columnName]
["max"]

```

descriptive

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p
\						
Mean	108.0	67.303395	66.333163	66.370186	72.100558	62.278186
Median	108.0	67.0	65.0	66.0	71.0	62.0
Mode	1	62.0	63.0	65.0	60.0	56.7
Q1:25%	54.5	60.6	60.9	61.0	60.0	57.945
Q2:50%	108.0	67.0	65.0	66.0	71.0	62.0
Q3:75%	161.5	75.7	73.0	72.0	83.5	66.255
99%	212.86	87.0	91.86	83.86	97.0	76.1142
Q4:100%	215.0	89.4	97.7	91.0	98.0	77.89

	salary
Mean	288655.405405
Median	265000.0
Mode	300000.0
Q1:25%	240000.0
Q2:50%	265000.0
Q3:75%	300000.0
99%	NaN
Q4:100%	940000.0

IQR

```

descriptive=pd.DataFrame(index=["Mean","Median","Mode","Q1:25%","Q2:50%","Q3:75%","99%","Q4:100%","IQR","1.5rule","Lesser","Greater","Min","Max"],columns=quan)
for columnName in quan:

```

```

descriptive[columnName]["Mean"]=dataset[columnName].mean()
descriptive[columnName]["Median"]=dataset[columnName].median()
descriptive[columnName]["Mode"]=dataset[columnName].mode()[0]
descriptive[columnName]["Q1:25%"]=dataset.describe()[columnName]
["25%"]
descriptive[columnName]["Q2:50%"]=dataset.describe()[columnName]
["50%"]
descriptive[columnName]["Q3:75%"]=dataset.describe()[columnName]
["75%"]
descriptive[columnName]
["99%"]=np.percentile(dataset[columnName],99)
descriptive[columnName]["Q4:100%"]=dataset.describe()[columnName]
["max"]
descriptive[columnName]["IQR"]=descriptive[columnName]["Q3:75%"]-
descriptive[columnName]["Q1:25%"]
descriptive[columnName]["1.5rule"]=1.5*descriptive[columnName]
["IQR"]
descriptive[columnName]["Lesser"]=descriptive[columnName]
["Q1:25%"]-descriptive[columnName]["1.5rule"]
descriptive[columnName]["Greater"]=descriptive[columnName]
["Q3:75%"]+descriptive[columnName]["1.5rule"]
descriptive[columnName]["Min"]=dataset[columnName].min()
descriptive[columnName]["Max"]=dataset[columnName].max()

```

descriptive

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p
\						
Mean	108.0	67.303395	66.333163	66.370186	72.100558	62.278186
Median	108.0	67.0	65.0	66.0	71.0	62.0
Mode	1	62.0	63.0	65.0	60.0	56.7
Q1:25%	54.5	60.6	60.9	61.0	60.0	57.945
Q2:50%	108.0	67.0	65.0	66.0	71.0	62.0
Q3:75%	161.5	75.7	73.0	72.0	83.5	66.255
99%	212.86	87.0	91.86	83.86	97.0	76.1142
Q4:100%	215.0	89.4	97.7	91.0	98.0	77.89
IQR	107.0	15.1	12.1	11.0	23.5	8.31
1.5rule	160.5	22.65	18.15	16.5	35.25	12.465
Lesser	-106.0	37.95	42.75	44.5	24.75	45.48

Greater	322.0	98.35	91.15	88.5	118.75	78.72
Min	1	40.89	37.0	50.0	50.0	51.21
Max	215	89.4	97.7	91.0	98.0	77.89

```

salary
Mean    288655.405405
Median   265000.0
Mode     300000.0
Q1:25%   240000.0
Q2:50%   265000.0
Q3:75%   300000.0
99%      NaN
Q4:100%  940000.0
IQR       60000.0
1.5rule   90000.0
Lesser    150000.0
Greater   390000.0
Min       200000.0
Max       940000.0

```

```
descriptive=pd.DataFrame(index=["Mean", "Median", "Mode", "Q1:25%", "Q2:50%",
```

```

"Q3:75%", "99%", "Q4:100%", "IQR", "1.5rule", "Lesser", "Greater", "Min", "Max"],
columns=quan)

```

```

for columnName in quan:
    descriptive[columnName]["Mean"]=dataset[columnName].mean()
    descriptive[columnName]["Median"]=dataset[columnName].median()
    descriptive[columnName]["Mode"]=dataset[columnName].mode()[0]
    descriptive[columnName]["Q1:25%"]=dataset.describe()[columnName]
    descriptive[columnName]["Q2:50%"]=dataset.describe()[columnName]
    descriptive[columnName]["Q3:75%"]=dataset.describe()[columnName]
    descriptive[columnName]
    descriptive[columnName]["99%"]=np.percentile(dataset[columnName], 99)
    descriptive[columnName]["Q4:100%"]=dataset.describe()[columnName]
    descriptive[columnName]
    descriptive[columnName]["IQR"]=descriptive[columnName]["Q3:75%"]-
descriptive[columnName]["Q1:25%"]
    descriptive[columnName]["1.5rule"]=1.5*descriptive[columnName]
    descriptive[columnName]
    descriptive[columnName]["Lesser"]=descriptive[columnName]

```

```

["Q1:25%"]-descriptive[columnName]["1.5rule"]
    descriptive[columnName]["Greater"]=descriptive[columnName]
["Q3:75%"]+descriptive[columnName]["1.5rule"]
    descriptive[columnName]["Min"]=dataset[columnName].min()
    descriptive[columnName]["Max"]=dataset[columnName].max()

```

descriptive

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p
\						
Mean	108.0	67.303395	66.333163	66.370186	72.100558	62.278186
Median	108.0	67.0	65.0	66.0	71.0	62.0
Mode	1	62.0	63.0	65.0	60.0	56.7
Q1:25%	54.5	60.6	60.9	61.0	60.0	57.945
Q2:50%	108.0	67.0	65.0	66.0	71.0	62.0
Q3:75%	161.5	75.7	73.0	72.0	83.5	66.255
99%	212.86	87.0	91.86	83.86	97.0	76.1142
Q4:100%	215.0	89.4	97.7	91.0	98.0	77.89
IQR	107.0	15.1	12.1	11.0	23.5	8.31
1.5rule	160.5	22.65	18.15	16.5	35.25	12.465
Lesser	-106.0	37.95	42.75	44.5	24.75	45.48
Greater	322.0	98.35	91.15	88.5	118.75	78.72
Min	1	40.89	37.0	50.0	50.0	51.21
Max	215	89.4	97.7	91.0	98.0	77.89

	salary
Mean	288655.405405
Median	265000.0
Mode	300000.0
Q1:25%	240000.0
Q2:50%	265000.0
Q3:75%	300000.0
99%	NaN
Q4:100%	940000.0
IQR	60000.0
1.5rule	90000.0

Lesser	150000.0
Greater	390000.0
Min	200000.0
Max	940000.0

```
dir(dataset["ssc_p"])
```

```
['T',
 '_AXIS_LEN',
 '_AXIS_ORDERS',
 '_AXIS_TO_AXIS_NUMBER',
 '_HANDLED_TYPES',
 '__abs__',
 '__add__',
 '__and__',
 '__annotations__',
 '__array__',
 '__array_priority__',
 '__array_ufunc__',
 '__bool__',
 '__class__',
 '__column_consortium_standard__',
 '__contains__',
 '__copy__',
 '__deepcopy__',
 '__delattr__',
 '__delitem__',
 '__dict__',
 '__dir__',
 '__divmod__',
 '__doc__',
 '__eq__',
 '__finalize__',
 '__float__',
 '__floordiv__',
 '__format__',
 '__ge__',
 '__getattr__',
 '__getattribute__',
 '__getitem__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__iadd__',
 '__iand__',
 '__ifloordiv__',
 '__imod__',
 '__imul__',
 '__init__',
 '__init_subclass__']
```

```
'__int__',
'__invert__',
'__ior__',
'__ipow__',
'__isub__',
'__iter__',
'__itruediv__',
'__ixor__',
'__le__',
'__len__',
'__lt__',
'__matmul__',
'__mod__',
'__module__',
'__mul__',
'__ne__',
'__neg__',
'__new__',
'__nonzero__',
'__or__',
'__pandas_priority__',
'__pos__',
'__pow__',
'__radd__',
'__rand__',
'__rdivmod__',
'__reduce__',
'__reduce_ex__',
'__repr__',
'__rfloordiv__',
'__rmatmul__',
'__rmod__',
'__rmul__',
'__ror__',
'__round__',
'__rpow__',
'__rsub__',
'__rtruediv__',
'__rxor__',
'__setattr__',
'__setitem__',
'__setstate__',
'__sizeof__',
'__str__',
'__sub__',
'__subclasshook__',
'__truediv__',
'__weakref__',
'__xor__',
```

```
'_accessors',
'_accum_func',
'_agg_examples_doc',
'_agg_see_also_doc',
'_align_for_op',
'_align_frame',
'_align_series',
'_append',
'_arith_method',
'_as_manager',
'_attrs',
'_binop',
'_cacher',
'_can_hold_na',
'_check_inplace_and_allows_duplicate_labels',
'_check_is_chained_assignment_possible',
'_check_label_or_level_ambiguity',
'_check_setitem_copy',
'_clear_item_cache',
'_clip_with_one_bound',
'_clip_with_scalar',
'_cmp_method',
'_consolidate',
'_consolidate_inplace',
'_construct_axes_dict',
'_construct_result',
'_constructor',
'_constructor_expanddim',
'_constructor_expanddim_from_mgr',
'_constructor_from_mgr',
'_convert_dtypes',
'_data',
'_deprecate_downcast',
'_dir_additions',
'_dir_deletions',
'_drop_axis',
'_drop_labels_or_levels',
'_duplicated',
'_expanddim_from_mgr',
'_find_valid_index',
'_flags',
'_flex_method',
'_from_mgr',
'_get_axis',
'_get_axis_name',
'_get_axis_number',
'_get_axis_resolvers',
'_get_block_manager_axis',
'_get_bool_data',
```



```
'_get_cacher',
'_get_cleaned_column_resolvers',
'_get_index_resolvers',
'_get_label_or_level_values',
'_get_numeric_data',
'_get_rows_with_mask',
'_get_value',
'_get_values_tuple',
'_get_with',
'_getitem_slice',
'_getitem',
'_hidden_attrs',
'_indexed_same',
'_info_axis',
'_info_axis_name',
'_info_axis_number',
'_init_dict',
'_init_mgr',
'_inplace_method',
'_internal_names',
'_internal_names_set',
'_is_cached',
'_is_copy',
'_is_label_or_level_reference',
'_is_label_reference',
'_is_level_reference',
'_is_mixed_type',
'_is_view',
'_item_cache',
'_ixs',
'_logical_func',
'_logical_method',
'_map_values',
'_maybe_update_cacher',
'_memory_usage',
'_metadata',
'_mgr',
'_min_count_stat_function',
'_name',
'_needs_reindex_multi',
'_pad_or_backfill',
'_protect_consolidate',
'_reduce',
'_references',
'_reindex_axes',
'_reindex_indexer',
'_reindex_multi',
'_reindex_with_indexers',
'_rename',
```

```
'_replace_single',
'_repr_data_resource_',
'_repr_latex_',
'_reset_cache',
'_reset_cacher',
'_set_as_cached',
'_set_axis',
'_set_axis_name',
'_set_axis_nocheck',
'_set_is_copy',
'_set_labels',
'_set_name',
'_set_value',
'_set_values',
'_set_with',
'_set_with_engine',
'_shift_with_freq',
'_slice',
'_stat_function',
'_stat_function_ddof',
'_take_with_is_copy',
'_to_latex_via_styler',
'_typ',
'_update_inplace',
'_validate_dtype',
'_values',
'_where',
'abs',
'add',
'add_prefix',
'add_suffix',
'agg',
'aggregate',
'align',
'all',
'any',
'apply',
'argmax',
'argmin',
'argsort',
'array',
'asfreq',
'asof',
'astype',
'at',
'at_time',
'attrs',
'autocorr',
'axes',
```

```
'backfill',  
'between',  
'between_time',  
'bfill',  
'bool',  
'clip',  
'combine',  
'combine_first',  
'compare',  
'convert_dtypes',  
'copy',  
'corr',  
'count',  
'cov',  
'cummax',  
'cummin',  
'cumprod',  
'cumsum',  
'describe',  
'diff',  
'div',  
'divide',  
'divmod',  
'dot',  
'drop',  
'drop_duplicates',  
'droplevel',  
'dropna',  
'dtype',  
'dtypes',  
'duplicated',  
'empty',  
'eq',  
'equals',  
'ewm',  
'expanding',  
'explode',  
'factorize',  
'ffill',  
'fillna',  
'filter',  
'first',  
'first_valid_index',  
'flags',  
'floordiv',  
'ge',  
'get',  
'groupby',  
'gt',
```

```
'hasnans',
'head',
'hist',
'iat',
'idxmax',
'idxmin',
'iloc',
'index',
'infer_objects',
'info',
'interpolate',
'is_monotonic_decreasing',
'is_monotonic_increasing',
'is_unique',
'isin',
'isna',
'isnull',
'item',
'items',
'keys',
'kurt',
'kurtosis',
'last',
'last_valid_index',
'le',
'loc',
'lt',
'map',
'mask',
'max',
'mean',
'median',
'memory_usage',
'min',
'mod',
'mode',
'mul',
'multiply',
'name',
'nbytes',
'ndim',
'ne',
'nlargest',
'notna',
'notnull',
'nsmallest',
'nunique',
'pad',
'pct_change',
```

```
'pipe',  
'plot',  
'pop',  
'pow',  
'prod',  
'product',  
'quantile',  
'radd',  
'rank',  
'ravel',  
'rdiv',  
'rdivmod',  
'reindex',  
'reindex_like',  
'rename',  
'rename_axis',  
'reorder_levels',  
'repeat',  
'replace',  
'resample',  
'reset_index',  
'rfloordiv',  
'rmod',  
'rmul',  
'rolling',  
'round',  
'rpow',  
'rsub',  
'rtruediv',  
'sample',  
'searchsorted',  
'sem',  
'set_axis',  
'set_flags',  
'shape',  
'shift',  
'size',  
'skew',  
'sort_index',  
'sort_values',  
'squeeze',  
'std',  
'sub',  
'subtract',  
'sum',  
'swapaxes',  
'swaplevel',  
'tail',  
'take',
```

```
'to_clipboard',
'to_csv',
'to_dict',
'to_excel',
'to_frame',
'to_hdf',
'to_json',
'to_latex',
'to_list',
'to_markdown',
'to_numpy',
'to_period',
'to_pickle',
'to_sql',
'to_string',
'to_timestamp',
'to_xarray',
'transform',
'transpose',
'truediv',
'truncate',
'tz_convert',
'tz_localize',
'unique',
'unstack',
'update',
'value_counts',
'values',
'var',
'view',
'where',
'xs']
```

Finding Outliers Column

```
descriptive["degree_p"]["Min"]<descriptive["degree_p"]["Lesser"]
False
Lesser=[]
Greater=[]

for columnName in quan:
    if (descriptive[columnName]["Min"]<descriptive[columnName]
["Lesser"]):
        Lesser.append(columnName)
    if (descriptive[columnName]["Max"]>descriptive[columnName]
["Greater"]):
        Greater.append(columnName)
```

```
Lesser
['hsc_p']
Greater
['hsc_p', 'degree_p', 'salary']
```

Replacing Outlier if exists

```
for columnName in Lesser:
    dataset[columnName][dataset[columnName]<descriptive[columnName]
["Lesser"]]=descriptive[columnName]["Lesser"]
for columnName in Greater:
    dataset[columnName][dataset[columnName]<descriptive[columnName]
["Greater"]]=descriptive[columnName]["Greater"]
```

```
C:\Users\Maheshwaran\AppData\Local\Temp\
ipykernel_5340\4058271768.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
dataset[columnName][dataset[columnName]<descriptive[columnName]
["Lesser"]]=descriptive[columnName]["Lesser"]
```

```
C:\Users\Maheshwaran\AppData\Local\Temp\
ipykernel_5340\4058271768.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
dataset[columnName][dataset[columnName]<descriptive[columnName]
["Greater"]]=descriptive[columnName]["Greater"]
```

```
C:\Users\Maheshwaran\AppData\Local\Temp\
ipykernel_5340\4058271768.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
dataset[columnName][dataset[columnName]<descriptive[columnName]
["Greater"]]=descriptive[columnName]["Greater"]
```

```
C:\Users\Maheshwaran\AppData\Local\Temp\
ipykernel_5340\4058271768.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation:

https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
dataset[columnName][dataset[columnName]<descriptive[columnName]
["Greater"]]=descriptive[columnName]["Greater"]
```

```
def Univariate(dataset,quan):
```

```
descriptive=pd.DataFrame(index=["Mean","Median","Mode","Q1:25%","Q2:50%",
```

```
"Q3:75%","99%","Q4:100%","IQR","1.5rule","Lesser","Greater","Min","Max"],columns=quan)
```

```
    for columnName in quan:
        descriptive[columnName]["Mean"]=dataset[columnName].mean()
        descriptive[columnName]["Median"]=dataset[columnName].median()
        descriptive[columnName]["Mode"]=dataset[columnName].mode()[0]
        descriptive[columnName]["Q1:25%"]=dataset.describe()
[columnName]["25%"]
        descriptive[columnName]["Q2:50%"]=dataset.describe()
[columnName]["50%"]
        descriptive[columnName]["Q3:75%"]=dataset.describe()
[columnName]["75%"]
        descriptive[columnName]
["99%"]=np.percentile(dataset[columnName],99)
        descriptive[columnName]["Q4:100%"]=dataset.describe()
[columnName]["max"]
        descriptive[columnName]["IQR"]=descriptive[columnName]
["Q3:75%"]-descriptive[columnName]["Q1:25%"]
        descriptive[columnName]["1.5rule"]=1.5*descriptive[columnName]
["IQR"]
        descriptive[columnName]["Lesser"]=descriptive[columnName]
["Q1:25%"]-descriptive[columnName]["1.5rule"]
        descriptive[columnName]["Greater"]=descriptive[columnName]
["Q3:75%"]+descriptive[columnName]["1.5rule"]
        descriptive[columnName]["Min"]=dataset[columnName].min()
        descriptive[columnName]["Max"]=dataset[columnName].max()
    return descriptive
```

```
descriptive
```

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p
\						
Mean	108.0	67.303395	66.333163	66.370186	72.100558	62.278186
Median	108.0	67.0	65.0	66.0	71.0	62.0
Mode	1	62.0	63.0	65.0	60.0	56.7
Q1:25%	54.5	60.6	60.9	61.0	60.0	57.945

Q2:50%	108.0	67.0	65.0	66.0	71.0	62.0
Q3:75%	161.5	75.7	73.0	72.0	83.5	66.255
99%	212.86	87.0	91.86	83.86	97.0	76.1142
Q4:100%	215.0	89.4	97.7	91.0	98.0	77.89
IQR	107.0	15.1	12.1	11.0	23.5	8.31
1.5rule	160.5	22.65	18.15	16.5	35.25	12.465
Lesser	-106.0	37.95	42.75	44.5	24.75	45.48
Greater	322.0	98.35	91.15	88.5	118.75	78.72
Min	1	40.89	37.0	50.0	50.0	51.21
Max	215	89.4	97.7	91.0	98.0	77.89

```

salary
Mean      288655.405405
Median    265000.0
Mode      300000.0
Q1:25%    240000.0
Q2:50%    265000.0
Q3:75%    300000.0
99%       NaN
Q4:100%   940000.0
IQR       60000.0
1.5rule   90000.0
Lesser    150000.0
Greater   390000.0
Min       200000.0
Max       940000.0

```

```

Lesser=[]
Greater=[]

```

```

for columnName in quan:
    if (descriptive[columnName]["Min"]<descriptive[columnName]
["Lesser"]):
        Lesser.append(columnName)
    if (descriptive[columnName]["Max"]>descriptive[columnName]
["Greater"]):
        Greater.append(columnName)

```

```

Lesser

```

```
['hsc_p']
Greater
['hsc_p', 'degree_p', 'salary']
```

Frequency, Relative Frequency and Cumulative Relative Frequency

```
dataset["ssc_p"].value_counts()

ssc_p
62.00    11
63.00    10
67.00     9
52.00     9
73.00     9
..
69.70     1
80.92     1
83.00     1
86.50     1
80.60     1
Name: count, Length: 103, dtype: int64

freqTable=pd.DataFrame(columns=["Unique_Values","Frequency","Relative
Frequency","Cusum"])

freqTable

Empty DataFrame
Columns: [Unique_Values, Frequency, Relative Frequency, Cusum]
Index: []

freqTable["Unique_Values"]=dataset["ssc_p"].value_counts().index
freqTable["Frequency"]=dataset["ssc_p"].value_counts().values
freqTable["Relative Frequency"]=(freqTable["Frequency"]/103)
freqTable["Cusum"]=freqTable["Relative Frequency"].cumsum()

freqTable
```

	Unique_Values	Frequency	Relative Frequency	Cusum
0	62.00	11	0.106796	0.106796
1	63.00	10	0.097087	0.203883
2	67.00	9	0.087379	0.291262
3	52.00	9	0.087379	0.378641
4	73.00	9	0.087379	0.466019
..	...	...	...	...

98	69.70	1	0.009709	2.048544
99	80.92	1	0.009709	2.058252
100	83.00	1	0.009709	2.067961
101	86.50	1	0.009709	2.077670
102	80.60	1	0.009709	2.087379

[103 rows x 4 columns]

```
def freqTable(columnName,dataset):
```

```
    freqTable=pd.DataFrame(columns=["Unique_Values","Frequency","Relative  
Frequency","Cusum"])
```

```
    freqTable["Unique_Values"]=dataset[columnName].value_counts().index
    freqTable["Frequency"]=dataset[columnName].value_counts().values
    freqTable["Relative Frequency"]=(freqTable["Frequency"]/103)
    freqTable["Cusum"]=freqTable["Relative Frequency"].cumsum()
    return freqTable
```

```
freqTable("hsc_p",dataset)
```

	Unique_Values	Frequency	Relative Frequency	Cusum
0	390000.0	133	1.291262	1.291262
1	400000.0	4	0.038835	1.330097
2	500000.0	3	0.029126	1.359223
3	425000.0	1	0.009709	1.368932
4	393000.0	1	0.009709	1.378641
5	411000.0	1	0.009709	1.388350
6	450000.0	1	0.009709	1.398058
7	420000.0	1	0.009709	1.407767
8	940000.0	1	0.009709	1.417476
9	690000.0	1	0.009709	1.427184
10	650000.0	1	0.009709	1.436893

```
def Univariate(dataset,quan):
```

```
    descriptive=pd.DataFrame(index=["Mean","Median","Mode","Q1:25%","Q2:50  
%"],
```

```
    ["Q3:75%","99%","Q4:100%","IQR","1.5rule","Lesser","Greater","Min","Max  
"],columns=quan)
```

```
    for columnName in quan:
        descriptive[columnName]["Mean"]=dataset[columnName].mean()
        descriptive[columnName]["Median"]=dataset[columnName].median()
        descriptive[columnName]["Mode"]=dataset[columnName].mode()[0]
        descriptive[columnName]["Q1:25%"]=dataset.describe()  
[columnName]["25%"]
        descriptive[columnName]["Q2:50%"]=dataset.describe()  
[columnName]["50%"]
        descriptive[columnName]["Q3:75%"]=dataset.describe()  
[columnName]["75%"]
```

```

descriptive[columnName]
["99%"]=np.percentile(dataset[columnName],99)
    descriptive[columnName]["Q4:100%"]=dataset.describe()
[columnName]["max"]
    descriptive[columnName]["IQR"]=descriptive[columnName]
["Q3:75%"]-descriptive[columnName]["Q1:25%"]
    descriptive[columnName]["1.5rule"]=1.5*descriptive[columnName]
["IQR"]
    descriptive[columnName]["Lesser"]=descriptive[columnName]
["Q1:25%"]-descriptive[columnName]["1.5rule"]
    descriptive[columnName]["Greater"]=descriptive[columnName]
["Q3:75%"]+descriptive[columnName]["1.5rule"]
    descriptive[columnName]["Min"]=dataset[columnName].min()
    descriptive[columnName]["Max"]=dataset[columnName].max()
return descriptive

```

Univariate(dataset, quan)

	sl_no	ssc_p	hsc_p	degree_p	etest_p	mba_p
Mean	108.0	67.303395	91.211628	88.511628	72.100558	62.278186
Median	108.0	67.0	91.15	88.5	71.0	62.0
Mode	1	62.0	91.15	88.5	60.0	56.7
Q1:25%	54.5	60.6	91.15	88.5	60.0	57.945
Q2:50%	108.0	67.0	91.15	88.5	71.0	62.0
Q3:75%	161.5	75.7	91.15	88.5	83.5	66.255
99%	212.86	87.0	91.881	88.5	97.0	76.1142
Q4:100%	215.0	89.4	97.7	91.0	98.0	77.89
IQR	107.0	15.1	0.0	0.0	23.5	8.31
1.5rule	160.5	22.65	0.0	0.0	35.25	12.465
Lesser	-106.0	37.95	91.15	88.5	24.75	45.48
Greater	322.0	98.35	91.15	88.5	118.75	78.72
Min	1	40.89	91.15	88.5	50.0	51.21
Max	215	89.4	97.7	91.0	98.0	77.89

	salary
Mean	401006.756757

Median	390000.0
Mode	390000.0
Q1:25%	390000.0
Q2:50%	390000.0
Q3:75%	390000.0
99%	NaN
Q4:100%	940000.0
IQR	0.0
1.5rule	0.0
Lesser	390000.0
Greater	390000.0
Min	390000.0
Max	940000.0

Histogram, Skewness and Kurtosis

```
dataset["ssc_p"].skew()
```

```
-0.13264949031456927
```

```
dataset["ssc_p"].kurtosis()
```

```
-0.6075102066986688
```

```
def Univariate(dataset, quan):
```

```
    descriptive=pd.DataFrame(index=["Mean", "Median", "Mode", "Q1:25%", "Q2:50%",
```

```
                                "Q3:75%", "99%", "Q4:100%", "IQR", "1.5rule", "Lesser", "Greater",
```

```
                                "Min", "Max", "kurtosis", "skew"], columns=quan)
```

```
    for columnName in quan:
```

```
        descriptive[columnName]["Mean"]=dataset[columnName].mean()
```

```
        descriptive[columnName]["Median"]=dataset[columnName].median()
```

```
        descriptive[columnName]["Mode"]=dataset[columnName].mode()[0]
```

```
        descriptive[columnName]["Q1:25%"]=dataset.describe()
```

```
        [columnName]["25%"]
```

```
        descriptive[columnName]["Q2:50%"]=dataset.describe()
```

```
        [columnName]["50%"]
```

```
        descriptive[columnName]["Q3:75%"]=dataset.describe()
```

```
        [columnName]["75%"]
```

```
        descriptive[columnName]
```

```
        ["99%"]=np.percentile(dataset[columnName], 99)
```

```
        descriptive[columnName]["Q4:100%"]=dataset.describe()
```

```
        [columnName]["max"]
```

```
        descriptive[columnName]["IQR"]=descriptive[columnName]
```

```
        ["Q3:75%"]-descriptive[columnName]["Q1:25%"]
```

```
        descriptive[columnName]["1.5rule"]=1.5*descriptive[columnName]
```

```

["IQR"]
    descriptive[columnName]["Lesser"]=descriptive[columnName]
["Q1:25%"]-descriptive[columnName]["1.5rule"]
    descriptive[columnName]["Greater"]=descriptive[columnName]
["Q3:75%"]+descriptive[columnName]["1.5rule"]
    descriptive[columnName]["Min"]=dataset[columnName].min()
    descriptive[columnName]["Max"]=dataset[columnName].max()
    descriptive[columnName]
["kurtosis"]=dataset[columnName].kurtosis()
    descriptive[columnName]["skew"]=dataset[columnName].skew()
    return descriptive

```

Univariate(dataset,quan)

	sl_no	ssc_p	hsc_p	degree_p	etest_p
mba_p \					
Mean	108.0	67.303395	91.211628	88.511628	72.100558
62.278186					
Median	108.0	67.0	91.15	88.5	71.0
62.0					
Mode	1	62.0	91.15	88.5	60.0
56.7					
Q1:25%	54.5	60.6	91.15	88.5	60.0
57.945					
Q2:50%	108.0	67.0	91.15	88.5	71.0
62.0					
Q3:75%	161.5	75.7	91.15	88.5	83.5
66.255					
99%	212.86	87.0	91.881	88.5	97.0
76.1142					
Q4:100%	215.0	89.4	97.7	91.0	98.0
77.89					
IQR	107.0	15.1	0.0	0.0	23.5
8.31					
1.5rule	160.5	22.65	0.0	0.0	35.25
12.465					
Lesser	-106.0	37.95	91.15	88.5	24.75
45.48					
Greater	322.0	98.35	91.15	88.5	118.75
78.72					
Min	1	40.89	91.15	88.5	50.0
51.21					
Max	215	89.4	97.7	91.0	98.0
77.89					
kurtosis	-1.2	-0.60751	104.345545	215.0	-1.08858
0.470723					
skew	0.0	-0.132649	10.211327	14.662878	0.282308
0.313576					

salary

Mean	401006.756757
Median	390000.0
Mode	390000.0
Q1:25%	390000.0
Q2:50%	390000.0
Q3:75%	390000.0
99%	NaN
Q4:100%	940000.0
IQR	0.0
1.5rule	0.0
Lesser	390000.0
Greater	390000.0
Min	390000.0
Max	940000.0
kurtosis	59.102562
skew	7.241223

Standard Deviation and Variance

```
dataset["ssc_p"].var()
117.22837673549229

dataset["ssc_p"].std()
10.827205398231452

def Univariate(dataset, quan):

    descriptive=pd.DataFrame(index=["Mean", "Median", "Mode", "Q1:25%", "Q2:50%",
    "Q3:75%", "99%", "Q4:100%", "IQR", "1.5rule", "Lesser", "Greater",
    "Min", "Max", "kurtosis", "skew", "Var", "Std"], columns=quan)
    for columnName in quan:
        descriptive[columnName]["Mean"]=dataset[columnName].mean()
        descriptive[columnName]["Median"]=dataset[columnName].median()
        descriptive[columnName]["Mode"]=dataset[columnName].mode()[0]
        descriptive[columnName]["Q1:25%"]=dataset.describe()
        [columnName]["25%"]
        descriptive[columnName]["Q2:50%"]=dataset.describe()
        [columnName]["50%"]
        descriptive[columnName]["Q3:75%"]=dataset.describe()
        [columnName]["75%"]
        descriptive[columnName]
        ["99%"]=np.percentile(dataset[columnName], 99)
        descriptive[columnName]["Q4:100%"]=dataset.describe()
        [columnName]["max"]
```

```

        descriptive[columnName]["IQR"]=descriptive[columnName]
["Q3:75%"]-descriptive[columnName]["Q1:25%"]
        descriptive[columnName]["1.5rule"]=1.5*descriptive[columnName]
["IQR"]
        descriptive[columnName]["Lesser"]=descriptive[columnName]
["Q1:25%"]-descriptive[columnName]["1.5rule"]
        descriptive[columnName]["Greater"]=descriptive[columnName]
["Q3:75%"]+descriptive[columnName]["1.5rule"]
        descriptive[columnName]["Min"]=dataset[columnName].min()
        descriptive[columnName]["Max"]=dataset[columnName].max()
        descriptive[columnName]
["kurtosis"]=dataset[columnName].kurtosis()
        descriptive[columnName]["skew"]=dataset[columnName].skew()
        descriptive[columnName]["Var"]=dataset[columnName].var()
        descriptive[columnName]["Std"]=dataset[columnName].std()
    return descriptive

```

Univariate(dataset,quan)

	sl_no	ssc_p	hsc_p	degree_p	etest_p
mba_p \					
Mean	108.0	67.303395	91.211628	88.511628	72.100558
62.278186					
Median	108.0	67.0	91.15	88.5	71.0
62.0					
Mode	1	62.0	91.15	88.5	60.0
56.7					
Q1:25%	54.5	60.6	91.15	88.5	60.0
57.945					
Q2:50%	108.0	67.0	91.15	88.5	71.0
62.0					
Q3:75%	161.5	75.7	91.15	88.5	83.5
66.255					
99%	212.86	87.0	91.881	88.5	97.0
76.1142					
Q4:100%	215.0	89.4	97.7	91.0	98.0
77.89					
IQR	107.0	15.1	0.0	0.0	23.5
8.31					
1.5rule	160.5	22.65	0.0	0.0	35.25
12.465					
Lesser	-106.0	37.95	91.15	88.5	24.75
45.48					
Greater	322.0	98.35	91.15	88.5	118.75
78.72					
Min	1	40.89	91.15	88.5	50.0
51.21					
Max	215	89.4	97.7	91.0	98.0
77.89					
kurtosis	-1.2	-0.60751	104.345545	215.0	-1.08858
-					

0.470723					
skew	0.0	-0.132649	10.211327	14.662878	0.282308
0.313576					
Var	3870.0	117.228377	0.359958	0.02907	176.251018
34.028376					
Std	62.209324	10.827205	0.599965	0.170499	13.275956
5.833385					

	salary
Mean	401006.756757
Median	390000.0
Mode	390000.0
Q1:25%	390000.0
Q2:50%	390000.0
Q3:75%	390000.0
99%	NaN
Q4:100%	940000.0
IQR	0.0
1.5rule	0.0
Lesser	390000.0
Greater	390000.0
Min	390000.0
Max	940000.0
kurtosis	59.102562
skew	7.241223
Var	3299625804.375812
Std	57442.369418